Rockchip RK3506 RT-Thread SDK Quick Start Guide

ID: RK-JC-YF-A34

Release Version: V1.1.0

Date: 2024-11-29

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2024. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily describes the basic usage methods of the RK3506 SMP RT-Thread SDK, aiming to help developers quickly understand and use the RK3506 SDK development package.

Support Status for Each Chip System

Chipset	RT-Thread Version
RK3506	4.1

Intended Audience

This document (this guide) is mainly intended for:

Technical Support Engineers

Software Development Engineers

Revision History

Date	Version	Author	Change Description
2024-10-22	V1.0.0	Jiahang Zheng	Initial Version
2024-11-29	V1.1.0	Jiahang Zheng	Add file system and OTA

Table of Contents

Rockchip RK3506 RT-Thread SDK Quick Start Guide

1. Setting Up the Development Environment
    - 1.1 Preparing the Development Environment
    - 1.2 Installing Libraries and Toolkits
        - 1.2.1 Checking and Upgrading the Host's `scons` Version
        - 1.2.2 Checking and Upgrading the Host's `python` Version
        - 1.2.3 Checking and Upgrading the Host's `make` Version
2. Software Development Guide
    - 2.1 Development Guide
    - 2.2 Chip Documentation
3. Hardware Development Guide
4. Directory Structure
5. Introduction to Cross-Compilation Toolchains for SDK
    - 5.1 U-Boot Compilation Toolchain
    - 5.2 RT-Thread Compilation Toolchain
6. SDK Compilation Instructions
    - 6.1 SDK Compilation Command Review
    - 6.2 SDK Board-Level Configuration
    - 6.3 Configuration File
        - 6.3.1 General Compilation Configuration File
    - 6.4 RT-Thread Firmware Packaging Configuration File
    - 6.5 Partition Table Configuration File
    - 6.6 General Compilation Command
    - 6.7 Special Compilation Command
        - 6.7.1 U-Boot Compilation Command
        - 6.7.2 RT-Thread Compilation Command
        - 6.7.3 root.img bundle
        - 6.7.4 RK HAL Compilation Command
7. RT-Thread Resource Configuration
    - 7.1 RT-Thread Configuration File
        - 7.1.1 Board-Level Related Files
        - 7.1.2 Compilation Related Files
    - 7.2 RT-Thread Memory Resources
        - 7.2.1 AP Runtime Memory Configuration
    - 7.3 Introduction to Compilation and Linking Scripts
        - 7.3.1 Overview
            - 7.3.1.1 Stack Size Allocation
            - 7.3.1.2 Heap Size Allocation
8. OTA
    - 8.1 Firmware Format
    - 8.2 configuration instruction
        - 8.2.1 parameter.txt configuration
        - 8.2.2 UBoot configuration
        - 8.2.3 RT-Thread configuration
    - 8.3 Version ID Modification
    - 8.4 A/B Firmware Boot Information
    - 8.5 OTA Trigger Methods
        - 8.5.1 Default Trigger Method
        - 8.5.2 Other Trigger Methods
9. Flashing Instructions
    - 9.1 Windows Flashing Instructions
    - 9.2 Linux Flashing Instructions
10. Debugging and Running

10.1 System Startup
10.2 Serial Port Debugging

1. Setting Up the Development Environment

1.1 Preparing the Development Environment

We recommend using a system with Ubuntu 22.04 or a higher version for compilation. In addition to system requirements, there are also other requirements in terms of hardware and software.
Hardware requirements: 64-bit system, disk space greater than 40G. If you are performing multiple builds, you will need more disk space.
Software requirements: System with Ubuntu 22.04 or a higher version.

1.2 Installing Libraries and Toolkits

When developing devices using the command line, you can install the libraries and tools required for compiling the SDK through the following steps.

Use the following apt-get command to install the libraries and tools needed for subsequent operations:

```
sudo apt-get update && sudo apt-get install git ssh make gcc python-is-python3
python2
```

> **Note:**
> The installation command is suitable for Ubuntu 22.04; for other versions, please use the corresponding installation commands based on the package names. If you encounter errors during compilation, you can install the corresponding software packages based on the error messages. Specifically:
>
> - Python requires the installation of version 3.6 or higher; this example uses Python 3.6.
> - Make requires the installation of version 4.0 or higher; this example uses Make 4.2.
> - Scons requires the installation of version 4.0.1 or higher.

1.2.1 Checking and Upgrading the Host's `scons` Version

Checking and upgrading the host's `scons` version can be done as follows:

- Check the host's `scons` version

```
$ scons --version
SCons by Steven Knight et al.:
        SCons: v4.0.1.c289977f8b34786ab6c334311e232886da7e8df1, 2020-07-17
01:50:03, by bdbaddog on ProDog2020
        SCons path: ['/usr/lib/python3/dist-packages/SCons']
Copyright (c) 2001 - 2020 The SCons Foundation
```

If the requirement of `scons>=4.0.1` version is not met, it can be upgraded as follows:

- Upgrade to the new version `SCons: v4.0.1`

```
wget https://sourceforge.net/projects/scons/files/scons/4.0.1/scons-4.0.1.zip
unzip scons-4.0.1.zip
cd SCons-4.0.1/
sudo python setup.py install
```

1.2.2 Checking and Upgrading the Host's `python` Version

The method to check and upgrade the host's `python` version is as follows:

- Checking the host `python` version

```
$ python3 --version
Python 3.10.6
```

If the requirement of python>=3.6 is not met, it can be upgraded as follows:

- Upgrading to the new version `python 3.6.15`

```
PYTHON3_VER=3.6.15
echo "wget
https://www.python.org/ftp/python/${PYTHON3_VER}/Python-${PYTHON3_VER}.tgz"
echo "tar xf Python-${PYTHON3_VER}.tgz"
echo "cd Python-${PYTHON3_VER}"
echo "sudo apt-get install libsqlite3-dev"
echo "./configure --enable-optimizations"
echo "sudo make install -j8"
```

1.2.3 Checking and Upgrading the Host's `make` Version

The method to check and upgrade the host's `make` version is as follows:

- Checking the host's `make` version

```
$ make -v
GNU Make 4.2
Built for x86_64-pc-linux-gnu
```

- Upgrading to a newer version of `make 4.2`

```
$ sudo apt update && sudo apt install -y autoconf autopoint

git clone https://gitee.com/mirrors/make.git
cd make
git checkout 4.2
git am $BUILDROOT_DIR/package/make/*.patch
autoreconf -f -i
./configure
make make -j8
sudo install -m 0755 make /usr/bin/make
```

2. Software Development Guide

2.1 Development Guide

The RT-Thread has two kinds of systems: SMP and AMP. Under AMP, RT-Thread only occupies one core, and the rest cores can run other systems such as Linux. This document only focuses on SMP RT-Thread. To assist development engineers in quickly getting started with the development and debugging work of RT-Thread SDK, module driver documents are released with the SDK. It can be obtained in the `docs/cn/Common` directory and will be continuously improved and updated.

2.2 Chip Documentation

To assist development engineers in quickly getting started with familiarizing themselves with the development and debugging of RK3506, the "Rockchip RK3506 Datasheet V0.1-20240516.pdf" chip manual is released along with the SDK. It can be obtained in the `docs/en/RK3506/Datasheet` directory, and will continue to be improved and updated.

3. Hardware Development Guide

Hardware-related development can refer to the user guide, located in the project directory:

RK3506 Hardware Design Guide:

```
<SDK>/docs/en/RK3506/Hardware/
```

4. Directory Structure

Below is an explanation of the main directories in the SDK:

```
.
├── app
│   └── rkadk
├── build.sh          # General compilation script for the SDK
├── device
│   └── Rockchip      # General compilation configuration and tools directory for
the SDK
├── docs              # Documentation related to the SDK
├── external          # Contains third-party related repositories
├── hal
│   ├── doc           # Documentation related to HAL
│   ├── html          # HAL doxygen-generated driver documentation
│   ├── lib           # HAL driver code
│   ├── project       # HAL project directory
```

```
│   ├── test        # HAL test code
│   └── tools       # Tools related to HAL
├── prebuilts       # SDK compilation toolchain (including compilation of U-Boot, HAL compilation toolchain)
├── rkbin           # SDK-related Binary files (including Loader, Trust, etc., used for packaging basic bin files)
├── rockdev         # Directory for generating images for the SDK
├── rtos            # Code related to RT-Thread
├── tools
│   ├── linux       # Burning tool for the Linux platform
│   ├── mac         # Burning tool for the Mac platform
│   ├── standalone  # Configuration parameters for burning tools
│   └── windows     # Burning tools and drivers for the Windows platform
└── u-boot          # Code related to U-boot
```

5. Introduction to Cross-Compilation Toolchains for SDK

Considering that the Rockchip RT-Thread SDK currently only compiles in a Linux PC environment, we have only provided cross-compilation toolchains for Linux. The compilation toolchains used by U-Boot and RT-Thread are different.

5.1 U-Boot Compilation Toolchain

The SDK prebuilts directory contains a pre-installed cross-compilation toolchain currently used for U-Boot compilation, as follows:

Directory	Description
prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf	GCC ARM 10.3.1 32-bit toolchain

5.2 RT-Thread Compilation Toolchain

Directory	Description
prebuilts/gcc/linux-x86/arm/gcc-arm-none-eabi-10-2020-q4-major-x86_64-linux	GCC ARM 32-bit toolchain

6. SDK Compilation Instructions

The SDK can be configured and compiled with `make` or `./build.sh` by adding target parameters for specific functionalities. For detailed compilation instructions, refer to `device/Rockchip/common/README.md`.

To ensure smooth SDK updates, it is recommended to clean up previous compilation artifacts before updating. This practice helps avoid potential compatibility issues or compilation errors, as old artifacts may not be compatible with the new version of the SDK. To clean these artifacts, simply run the command `./build.sh cleanall`.

One-Click Compilation

Chip	Type	Reference Model	One-Click Compilation
RK3506	Development Board	RK3506G EVB1	./build.sh lunch:rockchip_rk3506_g_evb1_smp_defconfig && ./build.sh

After compiling successfully, the following firmware is available under '/rockdev/' :

```
.
├── amp.img
├── MiniLoaderAll.bin -> ../../u-boot/rk3506_spl_loader_v1.02.110.bin
├── parameter.txt -> ../../device/rockchip/.chips/rk3506/parameter-128M-smp.txt
├── root.img
├── uboot.img -> ../../u-boot/uboot.img
└── update.img -> ../update/Image/update.img

0 directories, 6 files
```

> **Note**:
>
> - **U-boot Compilation**: It requires specifying a toolchain.
> - For example, the SDK includes a 32-bit toolchain:
>   export CROSS_COMPILE=/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf-
> - **RT-Thread Compilation**:
>   export RTT_EXEC_PATH=/prebuilts/gcc/linux-x86/arm/gcc-arm-none-eabi-10-2020-q4-major-x86_64-linux/bin/

6.1 SDK Compilation Command Review

`make help`, for example:

```
$ make help
  menuconfig            - interactive curses-based configurator
  oldconfig             - resolve any unresolved symbols in .config
  syncconfig            - Same as oldconfig, but quietly, additionally update
deps
  olddefconfig          - Same as syncconfig but sets new symbols to their
default value
  savedefconfig         - Save current config to RK_DEFCONFIG (minimal config)
  ...
```

The actual execution of make is `./build.sh`

That is, you can also compile related features by running `./build.sh <target>`. For specific compilation commands, you can view them through `./build.sh help`.

6.2 SDK Board-Level Configuration

Navigate to the project directory `<SDK>/device/Rockchip/rk3506`:

Board-Level Configuration	Description
rockchip_rk3506_g_evb1_smp_defconfig	Suitable for RK3506G EVB1 boards using the RT-Thread SMP scheme CPU0~CPU2: RT_Thread

6.3 Configuration File

Before compiling the RT-Thread SDK, please select and modify the relevant configuration files first.

6.3.1 General Compilation Configuration File

RT-Thread SDK Compilation Configuration: Primarily informs the compilation script of the project location for RT-Thread or Bare-metal, as well as the specific configurations of the accompanying projects. The default configuration for RT-Thread SDK is located at `<SDK>/device/rockchip/.chip/rockchip_rk3506_g_evb1_smp_defconfig`, with the following configurations:

```
RK_AMP=y                                        # Support for AMP RTOS / Bare-
metal
RK_AMP_FIT_ITS="smp.its"                        # RT-Thread firmware packaging
configuration file
RK_UBOOT_CFG_FRAGMENTS="rk-amp"                 # Add AMP U-Boot config
configuration file
RK_UBOOT_SPL=y                                  # Use SPL
RK_USE_FIT_IMG=y                                # Use FIT format for firmware
packaging
# RK_KERNEL is not set
RK_PARAMETER="parameter-128M-smp.txt"           # Partition table configuration
file
# Resource path to root.img, relative to rtos/bsp/rockchip/rk3506-32
RK_AMP_RTT_ROOT_DATA="../../../applications/lvgl_demo/rk_demo/resource"
RK_UPDATE=y                                     # Bundle update.img
```

6.4 RT-Thread Firmware Packaging Configuration File

RT-Thread firmware packaging uses the standard FIT format. The FIT format is natively supported and promoted by U-Boot. The RT-Thread SDK also parses ITS configuration files through scripts to complete the construction of the project. For details, please refer to the section **RT-Thread Resource Configuration** . The complete ITS file is as follows:

```
/*
 * Copyright (C) 2024 Rockchip Electronic Co.,Ltd
 *
 * Simple U-boot fit source file containing ATF/OP-TEE/U-Boot/dtb/MCU
 */

/dts-v1/;

/ {
    description = "FIT Image with RT-thread";
    #address-cells = <1>;

    images {

        amp0 {
            description  = "RTOS SMP";
            data         = /incbin/("rtt0.bin");
            type         = "firmware";
            compression  = "none";
            arch         = "arm";
            os           = "RTOS";
            cpu          = <0xf00>;  // mpidr
            thumb        = <0>;      // 0: arm or thumb2; 1: thumb
            hyp          = <0>;      // 0: el1/svc; 1: el2/hyp
            load         = <0x00100000>;
            compile {
                size     = <0x07f00000>;
                sys      = "rtt";
                core     = "ap";
                rtt_config = "board/evb1/smp_defconfig";
            };
            udelay       = <10000>; //delay U-Boot will delay 10000us before
entering the system
            hash {
                algo     = "sha256";
            };
        };
    };

    configurations {
        default = "conf";
        conf {
            description = "RK3506-rtt-smp";
            loadables = "amp0";

            signature {
                algo = "sha256,rsa2048";
                padding = "pss";
                key-name-hint = "dev";
                sign-images = "loadables";
```

```
            };
        };
    };
};
```

6.5 Partition Table Configuration File

During the development process, it is necessary to adjust the partition table configuration based on the actual storage medium capacity and the size of the AMPAK firmware.

Manually modify the partition table configuration file parameter.txt. The partition table configuration file is located at: `<AMPAK_SDK>/device/Rockchip/.chip/$RK_PARAMETER`. The format for adding partitions is `start@size(part_name)`, with the unit being sector (512 Byte). For example, device/Rockchip/rk3506/parameter-128M-smp.txt:

```
FIRMWARE_VER:8.1
MACHINE_MODEL:RK3506
MACHINE_ID:007
MANUFACTURER: RK3506
MAGIC: 0x5041524B
ATAG: 0x00200800
MACHINE: 3506
CHECK_MASK: 0x80
PWR_HLD: 0,0,A,0,1
TYPE: GPT
GROW_ALIGN: 0
CMDLINE:mtdparts=:0x00002000@0x00002000(uboot),0x00004000@0x00004000(amp),0x00020
000@0x00008000(rootfs),-@0x00028000(userdata:grow)
uuid:rootfs=614e0000-0000-4b53-8000-1d28000054a9
```

6.6 General Compilation Command

The general compilation command for RT-Thread SDK is as follows, supporting one-click compilation and packaging functions:

```
./build.sh chip         # Select the SoC platform
./build.sh lunch        # Select the default configuration file
./build.sh              # One-click compilation and packaging
./build.sh uboot        # Compile U-Boot separately
./build.sh amp          # Compile RT-Thread / Bare-metal separately
./build.sh cleanall     # Clean all
./build.sh help         # Get help
```

`./build.sh amp` will read the AMP firmware packaging configuration file smp.its, and automatically complete the compilation and packaging of amp.img.

6.7 Special Compilation Command

6.7.1 U-Boot Compilation Command

Use the following SDK command to compile:

```
./build.sh uboot
```

Enter into U-Boot, when U-Boot is compiled as a standalone component, for the RK3506 platform, the SDK includes tailored configurations for uboot, as explained below:

Configuration Name	Description
rk3506_defconfig	Basic configuration for RK3506
rk-amp.config	AMP sub-configuration for RK3506

Usage: ./make.sh base configuration + sub-configuration

Reference command as follows:

```
cd <SDK>/u-boot/
./make.sh CROSS_COMPILE=<SDK>/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf- rk3506 rk-amp --spl-new
```

6.7.2 RT-Thread Compilation Command

Use the following command within the SDK to compile:

```
./build.sh amp
```

The configuration file used for compiling RT-Thread within the SDK needs to be specified in smp.its. For reference, see the **AMP Firmware Packaging Configuration File** section. The default configuration file used in the SDK is located at `<SDK>/rtos/bsp/rockchip/rk3506-32/board/evb1/smp_defconfig`. The smp.its configuration is as follows:

```
rtt_config = "board/evb1/smp_defconfig"
```

Navigate to the RTOS directory for compilation:
When RT-Thread is compiled as a standalone component, the configuration file used is .config. The `scons --menuconfig` command reads the content from .config, and after modifications are saved, it automatically generates the rtconfig.h file. The reference command is as follows:

```
cd <SDK>/rtos/bsp/rockchip/rk3506-32/
./build.sh 0
./mkimage.sh
```

Customers can modify the configuration using `scons --menuconfig` in `<SDK>/rtos/bsp/rockchip/rk3506-32/`, and then replace `board/evb1/smp_defconfig` with .config. This allows direct use of `./build.sh amp` within the SDK to generate firmware with modified configurations. The reference operation is as follows:

```
cd <SDK>/rtos/bsp/rockchip/rk3506-32/
scons --menuconfig
# Save modifications
cp .config board/evb1/smp_defconfig
cd <SDK>/
./build.sh amp
```

In combination with the SDK to unify the compilation mode, RT-Thread can be configured and compiled as follows:

```
cd <SDK>/rtos/bsp/rockchip/rk3506-32        # Enter RT-Thread project
cp board/evb1/smp_defconfig .config         # Based on the default configuration
provided by the SDK
scons --menuconfig                          # Configure the project using
menuconfig
# After the configuration is completed, it is saved in .config, and rtconfig.h is
automatically generated
cp .config board/evb1/smp_defconfig         # You can override the default SDK
configuration and go back to the SDK root
cd <SDK>/
./build.sh
```

6.7.3 root.img bundle

RT-Thread supports FAT filesystems as described in **"Rockchip_Developer_Guide_RT-Thread_SPIFLASH_EN.pdf"**.

Use the following SDK command to compile:

```
# root.img is packaged when rt-threads are compiled
./build.sh amp
```

If `<SDK>/device/rockchip/.chip/` has defined RK_AMP_RTT_ROOT_DATA, `./build.sh` will generate the root.img under `<SDK>/rockdev`

```
# This is a based on the <SDK>/rtos/BSP/rockchip/rk3506-32 relative path
RK_AMP_RTT_ROOT_DATA="../../../applications/lvgl_demo/rk_demo/resource"
```

Instead of bundling root.img with './build.sh amp 'under the SDK, you can also use script bundling:

```
cd <SDK>/rtos/bsp/rockchip/rk3506-32/
# Option 1: Use./mkroot.sh script
# ./mkroot.sh root <resource_file> <parameter_file> <page_size_bytes>
<target_file>
# for example : nand flash page 2048 bytes, bundle resource/root generate
Image/root.img
./mkroot.sh root resource/root board/evb1/parameter-128M-smp.txt 2048
Image/root.img

# Option 2: Use./build.sh script
# ./build.sh root <resource_file> <PAGE_SIZE(bytes)>
# for example : nand page 2048 bytes, bundle resource/root generate
Image/root.img
./build.sh root resource/root 2048

# The mini ftl used by root is read-only "Rockchip_Developer_Guide_RT-
Thread_SPIFLASH_EN.pdf"
```

You can use the filesystem by following these steps:

1. scons --menuconfig enables filesystem support:

```
# SPI NAND Flash support
RT_USING_SNOR=n                         # disabled SPI Nor
RT_USING_MTD_NAND=y
RT_USING_SPINAND=y
RT_SPINAND_SPEED=80000000               # IO interface rate
RT_USING_SPINAND_FSPI_HOST=y
RT_USING_FTL=y
RT_USING_DHARA=y

# RT-Thread DFS support
CONFIG_RT_USING_DFS=y
CONFIG_DFS_USING_POSIX=y
CONFIG_DFS_USING_WORKDIR=y
CONFIG_DFS_FILESYSTEMS_MAX=4             # Number of filesystems that can be
mounted
CONFIG_DFS_FILESYSTEM_TYPES_MAX=4        # Number of filesystems that can be
mounted
CONFIG_DFS_FD_MAX=16
CONFIG_RT_USING_DFS_MNTTABLE=y           # To mount filesystems automatically at
boot, see mnt.c
CONFIG_RT_USING_DFS_ELMFAT=y             # FAT filesystem support

# elm-chan's FatFs, Generic FAT Filesystem Module
CONFIG_RT_DFS_ELM_CODE_PAGE=437
CONFIG_RT_DFS_ELM_WORD_ACCESS=y
CONFIG_RT_DFS_ELM_USE_LFN_3=y
CONFIG_RT_DFS_ELM_USE_LFN=3
CONFIG_RT_DFS_ELM_LFN_UNICODE_0=y
CONFIG_RT_DFS_ELM_LFN_UNICODE=0
CONFIG_RT_DFS_ELM_MAX_LFN=255
CONFIG_RT_DFS_ELM_DRIVES=3               # Number of FAT filesystems allowed to
mount
```

```
CONFIG_RT_DFS_ELM_MAX_SECTOR_SIZE=2048   # Set elm FatFs sector size to flash
pagesize, usually 2KB/4KB, refer to particle manual
CONFIG_RT_DFS_ELM_REENTRANT=y
CONFIG_RT_DFS_ELM_MUTEX_TIMEOUT=3000
CONFIG_RT_USING_DFS_DEVFS=y

CONFIG_RT_USING_USER_DATA_PART=y         # Without pre-burning userdata.img, you
can enable this configuration to automatically format the userdata partition and
mount it under /data
```

Burn firmware, successful mount can be viewed using 'ls'

```
msh />ls
Directory /:
data                <DIR>
udisk               <DIR>
sdcard              <DIR>
demo.txt            24
```

6.7.4 RK HAL Compilation Command

When compiling the RK HAL as a standalone component, refer to the following command:

```
cd <AMPAK_SDK>/hal/project/rk3506/GCC
./build.sh <cpu_id 0~3 or all>
cd ..
./mkimage.sh

make -j8
make clean
```

7. RT-Thread Resource Configuration

7.1 RT-Thread Configuration File

Typically, in chip-level projects, several board-level configurations are preset. During project usage, the `CONFIG_RT_BOARD_NAME` configuration can be modified through the `scons --menuconfig` command to achieve the requirement of adapting the same project to multiple boards.

Therefore, the content of RT-Thread configuration includes:

```
<SDK>/rtos/bsp/rockchip/rk3506-32/
├── applications
├── board
│   ├── common                  # Common configuration
│   ├── Kconfig
│   ├── evb1                    # Board-level configuration
│   └── SConscript
├── build.sh                    # Build script, containing some build
configurations
└── ...
```

- Common configuration part: The basic configuration of the chip, which is the mandatory code for the project and serves all board-level projects.
- Board-level configuration part: Board-level configuration, which configures related functions for specific boards, such as GPIO, UART, I2C, etc.
- Build command: The build command can achieve system parameter passing during compilation, providing flexible compilation instructions. When there is a `build.sh` script, parameters can be directly defined in `build.sh`. Alternatively, after `exporting` specific build parameters, use the `scons` command to compile.

7.1.1 Board-Level Related Files

RT-Thread project resource configuration, including general configurations, as well as board-level configurations.

General configuration files include:

```
# General configuration
<SDK>/rtos/bsp/rockchip/rk3506-32/board/common/board_base.c
<SDK>/rtos/bsp/rockchip/rk3506-32/board/common/iomux_base.c
```

General configurations define the initialization sequence for hardware startup and default hardware resources while providing a large number of `RT_WEAK` definitions, facilitating users to make corresponding replacements in board-level configurations.

```
RT_WEAK const struct clk_init clk_inits[];            // Weak-defined structure,
can be redefined in board-level configuration
RT_WEAK  void rt_hw_iomux_config(void);               // Weak-defined function,
can be redefined in board-level configuration
// ...                                                // Weak-defined resources
for different chips are not consistent, not listed one by one

void rt_hw_board_init(void)
{
    // ... Initialization sequence, do not modify
}
```

Board-level configuration files include:

```
# Board-level configuration
<AMPAK_SDK>/rtos/bsp/rockchip/rk3506-32/board/evb1/board.c
<AMPAK_SDK>/rtos/bsp/rockchip/rk3506-32/board/evb1/iomux.c
```

In board-level configurations, redefine the required `RT_WEAK` resources.

```
// <AMPAK_SDK>/rtos/bsp/rockchip/rk3562-32/board/rk3562_evb1_lp4x/iomux.c

void rt_hw_iomux_config(void)
{
    // ...
}
```

7.1.2 Compilation Related Files

RT-Thread is compiled using `scons`, and the files involved in the compilation include:

```
cd <SDK>/rtos/bsp/rockchip/rk3506-32/

rk3506-32/rtconfig.h                # Configuration file used for compilation
rtconfig.py                         # Compilation script, determines the location
of the compilation chain, compilation command input parameters, etc.
SConscript                          # SCONS link file
SConstruct                          # SCONS link file

build.sh                            # Quick compilation script for RT-Thread AP
Core
```

7.2 RT-Thread Memory Resources

RT-Thread's memory allocation is assigned by the compiled link script.

```
<SDK>/rtos/bsp/Rockchip/rk3506-32/gcc_arm.ld.S
```

7.2.1 AP Runtime Memory Configuration

```
/* cat <SDK>/rtos/bsp/Rockchip/rk3506-32/gcc_arm.ld.S */

MEMORY
{
    SRAM   (rxw) : ORIGIN = 0xfff80000, LENGTH = 64K            /* SYSTEM SRAM */
    DRAM   (rxw) : ORIGIN = FIRMWARE_BASE, LENGTH = DRAM_SIZE    /* DRAM */
#ifndef RT_USING_SMP
    LINUX_RPMSG (rxw) : ORIGIN = LINUX_RPMSG_BASE, LENGTH = LINUX_RPMSG_SIZE
#endif
}

# cat <SDK>/rtos/bsp/Rockchip/rk3506-32/rtconfig.py
# Perform variable conversion, connecting the build.sh script and gcc_arm.ld.S
file
```

```
CFLAGS += ' -DFIRMWARE_BASE={a} -DDRAM_SIZE={b} -DLINUX_RPMSG_BASE={c} -
DLINUX_RPMSG_SIZE={d}'.format(a=PRMEM_BASE, b=PRMEM_SIZE, c=LINUX_RPMSG_BASE,
d=LINUX_RPMSG_SIZE)

/* cat <SDK>/rtos/bsp/Rockchip/rk3506-32/build.sh */

# TODO: Please plan the memory according to the actual needs of the project.
# Requiring 1M alignment.
CPU0_MEM_BASE=0x03900000
CPU1_MEM_BASE=0x03a00000
CPU2_MEM_BASE=0x03e00000
CPU0_MEM_SIZE=0x00100000
CPU1_MEM_SIZE=0x00100000
CPU2_MEM_SIZE=0x00100000
```

RT-Thread's memory configuration, located in `gcc_arm.ld.S`, for the convenience of compilation configuration, some parameters are transformed through `rtconfig.py` to `build.sh`, facilitating compilation modifications.

The address information configured in the AP Core, even the actual physical address information of DDR.

DRAM can be configured through parameters in `build.sh`:

```
CPU0_MEM_BASE=0x03900000
CPU0_MEM_SIZE=0x00100000
export RTT_PRMEM_BASE=$(eval echo \$CPU$1_MEM_BASE)             /* DRAM starting
position */
export RTT_PRMEM_SIZE=$(eval echo \$CPU$1_MEM_SIZE)             /* DRAM capacity
size */
```

The above variables correspond to the DRAM area in `gcc_arm.ld.S`:

```
DRAM   (rxw) : ORIGIN = FIRMWARE_BASE, LENGTH = DRAM_SIZE    /* DRAM */
```

Different variable names are transformed through `rtconfig.py`.

Note: For firmware compiled with the `./build.sh` command in the SDK, the size of DRAM is defined in smp.its. For specific details, you can directly refer to smp.its. Except for the first 1M space used by the system, the rest of the RAM can be used by RT-Thread. The configuration is as follows:

```
# loade is the starting address of DRAM, size is the size of DRAM, corresponding
to CPU0_MEM_BASE and CPU0_MEM_SIZE in ./build.sh, smp.its defines the DRAM
starting address as 1M, size 127M

amp0 {
            description  = "rtos-smp";
            data         = /incbin/("rtt0.bin");
            type         = "firmware";
            compression  = "none";
            arch         = "arm";
            os           = "rtos";
            cpu          = <0xf00>;  // mpidr
            thumb        = <0>;      // 0: arm or thumb2; 1: thumb
            hyp          = <0>;      // 0: el1/svc; 1: el2/hyp
```

```
            load         = <0x00100000>;
            compile {
                size     = <0x07f00000>;
                sys      = "rtt";
                core     = "ap";
                rtt_config = "board/evb1/smp_defconfig";
            };
            udelay       = <10000>;
            hash {
                algo     = "sha256";
            };
        };
    };
```

7.3 Introduction to Compilation and Linking Scripts

7.3.1 Overview

The RK3506 utilizes an ARM compilation and linking script to manage the memory allocation within the CPU. For detailed information and usage methods of the compilation and linking script, users can refer to the official ARM documentation and the linking script file. After the code is compiled, the corresponding code and data memory allocation segments will be generated according to the compilation and linking script. For reference:

Linking script related data segment description:

Name	Type	Address	Description
text	Code segment	0	Stores the code segment and other read-only data
zero.table	Zero table	Immediately after text	Stores the start and end addresses of the zeroed section
data	Data segment	Immediately after zero.table	Stores the read-write data segment
ttb	MMU table	Immediately after data (aligned to 16K)	MMU table, currently only configured with a first-level page table
share_mem	Shared memory	Immediately after ttb	Stores shared data
linux_share_mem	Linux shared memory	Immediately after share_mem	Stores Linux shared data
bss	BSS segment	Immediately after linux_share_mem	Zeroed data segment
heap	Heap	Immediately after bss segment	Heap, the remaining memory is allocated entirely to the heap

Name	Type	Address	Description
stack	Stack	Immediately after heap	Fixed size of 1KB, placed at the end of the AMPAK

7.3.1.1 Stack Size Allocation

In the linker script file gcc_arm.ld.S, for different modes, the stack size has independent configurations, and users can modify them according to their needs. References and descriptions are as follows:

```
                                # Stacks are independent under different modes,
hence the configurations are also independent
__STACK_SIZE = 0x400;           # Size of the main stack, i.e., the stack size
under our normal running mode
__FIQ_STACK_SIZE = 0x400;       # Size of the stack under FIQ mode
__IRQ_STACK_SIZE = 0x400;       # Size of the stack under IRQ mode
__SVC_STACK_SIZE = 0x1000;      # Size of the stack under SVC mode
__ABT_STACK_SIZE = 0x400;       # Size of the stack under ABORT exception
__UND_STACK_SIZE = 0x400;       # Size of the stack under undefined exception
__TTB_SIZE = 0x4000;            # Size of the MMU page table, please do not
change
```

7.3.1.2 Heap Size Allocation

The size of the heap is dynamically allocated based on actual user usage. It depends on the allocation of the stack size and the space used by the code. The calculation method for the heap size can be referred to as follows:

```
Heap Size = Total Memory Size - Stack Size - Space Used by User Code
```

In the linker script file gcc_arm.ld.S, the definition of heap allocation is as follows:

```
SECTIONS
{
    # ......
    . = ALIGN(16);
    __ALL_STACK_SIZE = __STACK_SIZE + __FIQ_STACK_SIZE + __IRQ_STACK_SIZE +
__SVC_STACK_SIZE + __ABT_STACK_SIZE + __UND_STACK_SIZE;
    __STACK_START = ORIGIN(DRAM) + LENGTH(DRAM) - __ALL_STACK_SIZE;
    __HEAP_SIZE = (__STACK_START - .);
    .heap :
    {
        PROVIDE(__heap_start = .);
        . += __HEAP_SIZE;               # The heap lies between bss and stack,
and its size is all the remaining space after the system deducts all overhead
        PROVIDE(__heap_end = .);
    } > DRAM

    .stack __STACK_START :
    {
        # ......
```

```
    } > DRAM
}
```

8. OTA

OTA (Over-the-Air) technology refers to the remote upgrading of device firmware or software through wireless communication, eliminating the need for traditional manual updates via cable connections. Device manufacturers can use OTA technology to push the latest firmware and software updates to a large number of devices distributed globally, addressing security vulnerabilities, providing new features, or enhancing device performance.

The RK3506 employs an A/B firmware scheme for OTA upgrades, meaning that while the A firmware is running, it can upgrade the B firmware, and upon successful upgrade, it switches to running the B firmware. This method has the following features:

- The device can continue to be used during the OTA process.
- If the firmware upgrade fails, it can roll back to the previous firmware and continue to operate.

8.1 Firmware Format

It has the following features:

1. `update.img` contains the valid firmware and is ultimately upgraded as a partitioned upgrade.
2. It supports GPT partitioning.

8.2 configuration instruction

8.2.1 parameter.txt configuration

The configuration information for the `update.img` firmware partition is called `parameter.txt`, and its contents are as follows:

```
FIRMWARE_VER:8.1
MACHINE_MODEL:RK3506
MACHINE_ID:007
MANUFACTURER: RK3506
MAGIC: 0x5041524B
ATAG: 0x00200800
MACHINE: 3506
CHECK_MASK: 0x80
PWR_HLD: 0,0,A,0,1
TYPE: GPT
GROW_ALIGN: 0

CMDLINE:mtdparts=:0x00000600@0x00000000(loader),0x00002000@0x00002000(uboot),0x00
000800@0x00004000(misc),0x00002000@0x00004800(amp_a),0x00002000@0x00006800(amp_b)
,0x00020000@0x00008800(root),-@0x00028800(userdata:grow)
uuid:rootfs=614e0000-0000-4b53-8000-1d28000054a9
```

```
# 0x00002000@0x00004800(amp_a) as an example
# Its format is PartSize@PartOffset(Name), with PartSize and PartOffset measured
in sectors, which is 512 bytes.
# Each partition information is separated by a comma, and the next partition's
PartOffset is the previous partition's PartOffset + PartSize.
# The last partition does not need to specify PartSize; the software will
automatically calculate it based on the flash size.
# The GPT partition table is stored in the first block of the NAND FLASH, so
0x00000600@0x00000000(loader) contains the gpt table and the loader firmware,
which will offset and skip gpt when upgrading
# The root can also perform an A/B partition, which can be modified by the
customer on demand
```

The firmware packaging configuration file is called `package-file`, which specifies which firmwares are to be packaged into `update.img`.

Note:

- The userdata partition must be placed last.
- OTA cannot support the upgrade of gpt partition table at present, pay attention to reserve enough partition space.
- For SPI NAND, partitions need to reserve additional blocks to prevent bad blocks from occurring. Small partitions should reserve an extra block (256KB), while larger partitions should reserve an additional 1MB.

8.2.2 UBoot configuration

If you need to use the A/B partition, enable A/B partition support in UBoot:

```
CONFIG_ANDROID_AB=y
CONFIG_AVB_LIBAVB=y
CONFIG_AVB_LIBAVB_AB=y
CONFIG_AVB_LIBAVB_ATX=y
CONFIG_AVB_LIBAVB_USER=y
CONFIG_RK_AVB_LIBAVB_USER=y
```

8.2.3 RT-Thread configuration

```
CONFIG_RT_USING_NEW_OTA=y               # Enable OTA
CONFIG_RT_USING_NEW_OTA_MODE_AB=y
CONFIG_RT_USING_AVB_LIBAVB_AB=y
CONFIG_RT_USING_FWANALYSIS=y
```

You can enable the file system and USB support, and put the upgrade package `update.img` in the file system. The file system refers to the `root.img packaging` section above. USB detailed instructions and configuration reference **"Rockchip_Developer_Guide_RT-Thread_USB_EN.pdf"**

USB configuration：

```
# Using USB
CONFIG_RT_USING_USB=y
CONFIG_RT_USING_USB_HOST=y
CONFIG_RT_USBH_MSTORAGE=y
CONFIG_UDISK_MOUNTPOINT="udisk"
CONFIG_RT_USBD_THREAD_STACK_SZ=4096

CONFIG_RT_USING_DWC2_USBH=y             # OTG0
CONFIG_RT_USING_DWC2_USBH1=y            # OTG1
```

8.3 Version ID Modification

The OTA program by default compares the version number of the currently running firmware with the version number of the `update.img` to determine whether to perform an upgrade. If the version numbers do not match, the upgrade will proceed; otherwise, it will not. This part of the code can be found in `rk_ota_fw_version_chk`, and customers can modify the logic themselves.

To modify the version number, follow these steps, Edit the `parameter.txt` file located in the board-level `board` directory, for example, `board/evb/parameter.txt`.

```
FIRMWARE_VER: 1.0.0
```

If the version number is the same, the upgrade will not be triggered.
Similar to the following log:

```
OTA: current fw_version 0x1000000 new fw_version 0x1000000
OTA: rk_ota_fw_version_chk: fw version is same
OTA: rk_ota_process: no need update
```

8.4 A/B Firmware Boot Information

A/B firmware boot information is stored in the `misc` partition, and the specific contents are as follows:

```
typedef struct fw_ab_slot_data_t
{
    /* Slot priority. Valid values range from 0 to AVB_AB_MAX_PRIORITY,
     * both inclusive with 1 being the lowest and AVB_AB_MAX_PRIORITY
     * being the highest. The special value 0 is used to indicate the
     * slot is unbootable.
     */
    uint8_t priority;

    /* Number of times left attempting to boot this slot ranging from 0
     * to AVB_AB_MAX_TRIES_REMAINING.
     */
    uint8_t tries_remaining;

    /* Non-zero if this slot has booted successfully, 0 otherwise. */
    uint8_t successful_boot;
```

```
    /* Reserved for future use. */
    uint8_t reserved[1];
} fw_ab_slot_data;
```

After upgrading using the PC tool, during the first boot, the loader program will initialize the A/B firmware boot information, as seen in the printed information below:

```
A/B-slot: _a, successful: 0, tries-remain: 7

# The first line above is the information for firmware A, and the second line is
for firmware B.
# Firmware A has a higher priority, indicating that it will boot from firmware A.
# The `tries_remaining` shows the number of remaining boot attempts for the
corresponding firmware, and `successful_boot` indicates whether the firmware has
booted successfully. If set to 1, the next reboot will continue booting from that
firmware. The printed information after firmware A successfully boots is as
follows:

A/B-slot: _a, successful: 1, tries-remain: 0
```

The `successful_boot` flag needs to be set by the application after RT-Thread starts. The default code for setting the successful boot status in the SDK can be found in `bsp/rockchip/rk3506-32/applications/main.c`.

```
// It is recommended to move this to a location that can indicate that the
business is successfully running
int main(void)
{
    ...
#ifdef RT_USING_NEW_OTA
    rk_ota_set_boot_success();
#endif
    ...
}
```

You can determine whether the currently running firmware is A or B by calling the function `int fw_slot_get_current_running(uint32_t *cur_slot)`. `cur_slot=0` indicates firmware A, while `cur_slot=1` indicates firmware B.

8.5 OTA Trigger Methods

The OTA code is located in the `components/ota/` directory, with the main program being `rk_ota_process` in `ota_new.c`.

8.5.1 Default Trigger Method

After booting, the system will look for `update.img` in the following directory to attempt the upgrade.

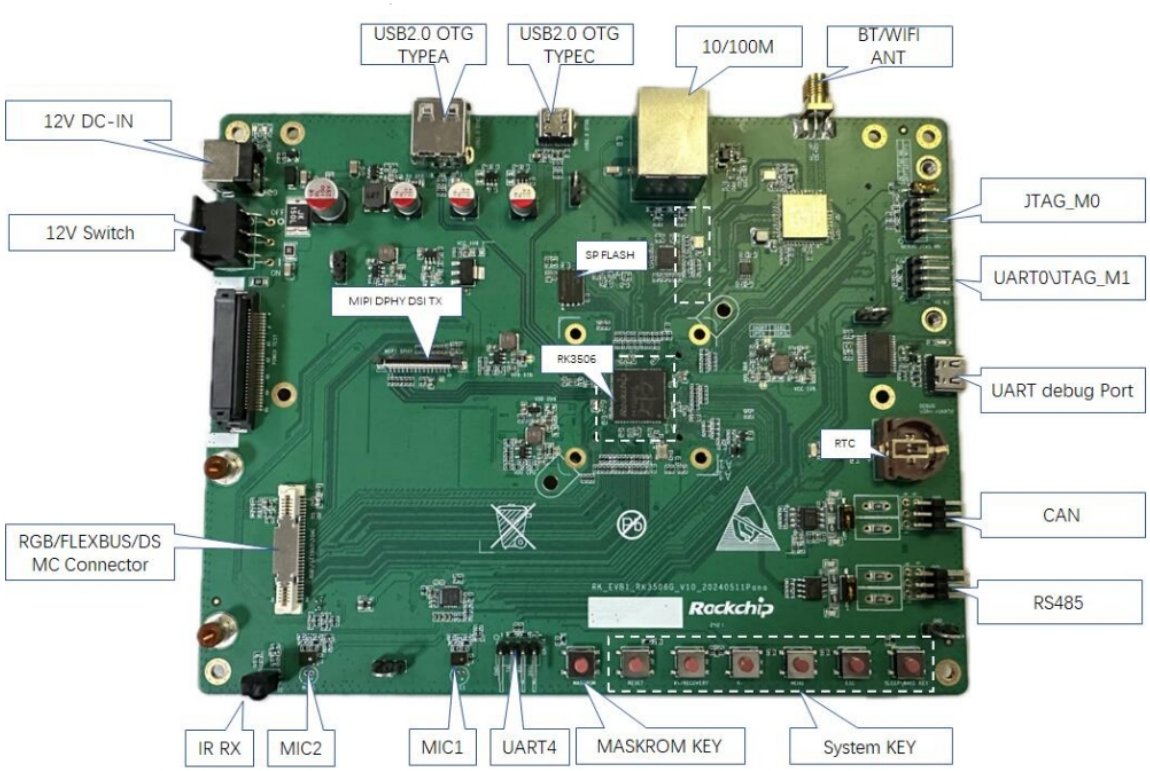
```
RT_WEAK const char *const ota_fw_path[] =
{
    "/",
    "/udisk",
    "/sdcard"
};
```

8.5.2 Other Trigger Methods

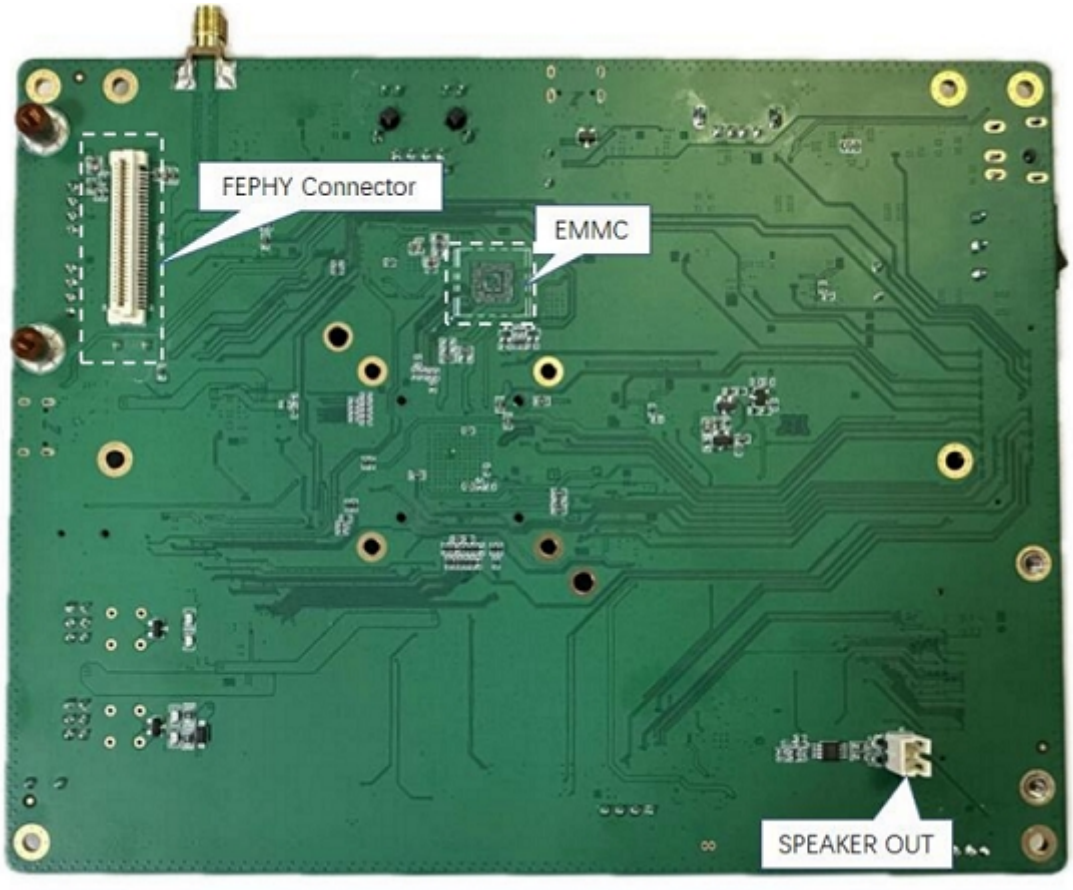
Other trigger methods, such as network downloads, currently require the customer to implement them themselves and then call `rk_ota_process` to initiate the upgrade.

9. Flashing Instructions

The interface layout of the TOP Surface of RK3506 EVB 1 development board is as follows:



The interface layout of the Bottom Surface of RK3506 EVB 1 development board is as follows:

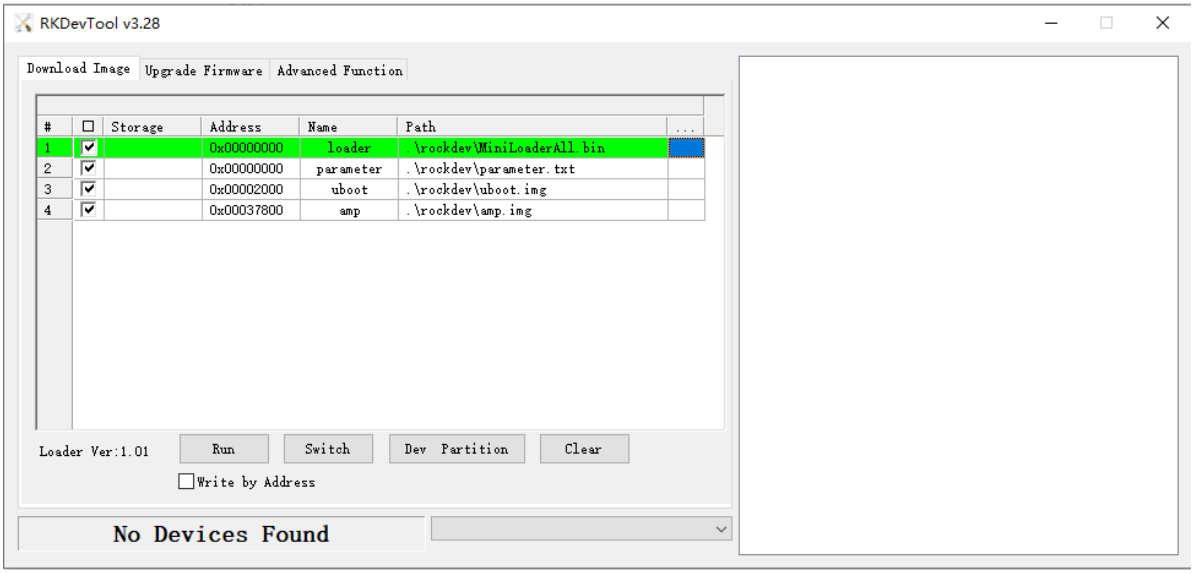


9.1 Windows Flashing Instructions

The SDK provides a Windows flashing tool (the tool version must be V3.28 or above), located in the root directory of the project:

```
tools/
├── windows/RKDevTool
```

As shown in the figure below, after compiling the corresponding firmware, the device needs to enter the MASKROM or BootROM flashing mode. Connect the USB download cable, hold the "MASKROM" button and press the reset button "RST" without releasing, then release to enter the MASKROM mode. Load the corresponding path of the compiled firmware and click "Execute" to flash. Alternatively, hold the "recovery" button without releasing and press the reset button "RST", then release to enter the loader mode for flashing. Below are the partition offsets for MASKROM mode and the flashing files.
(Note: The Windows PC needs to run the tool with administrator privileges to execute)



Note: Before flashing, the latest USB driver must be installed. For more details on the driver, see:

```
<SDK>/tools/windows/DriverAssitant_v5.13.zip
```

9.2 Linux Flashing Instructions

The flashing tool for Linux is located in the `tools/linux` directory (the Linux_Upgrade_Tool version must be V2.36 or above). Please ensure that your board is connected to MASKROM/loader rockusb. For example, if the firmware compiled is in the `rockdev` directory, the upgrade commands are as follows:

```
sudo ./upgrade_tool ul rockdev/MiniLoaderAll.bin -noreset
sudo ./upgrade_tool di -p rockdev/parameter.txt
sudo ./upgrade_tool di -u rockdev/uboot.img
sudo ./upgrade_tool di -amp rocdev/amp.img
sudo ./upgrade_tool rd
```

Or upgrade the complete firmware after packaging:

```
sudo ./upgrade_tool uf rockdev/update.img
```

Or upgrade in the root directory when the device is running in MASKROM state:

```
./rkflash.sh
```

10. Debugging and Running

10.1 System Startup

There are several methods to start the system:

1. Automatic restart after firmware upgrade;
2. Direct startup by inserting USB power supply;
3. For devices with battery power, press the Reset button to start;

Note: Different hardware designs will have different power-up and startup methods.

10.2 Serial Port Debugging

RK3506 RT-Thread SDK uses UART0 for printing by default on the main core.

Serial communication configuration information is as follows:

Baud rate: 1500000

Data bits: 8

Stop bits: 1

Parity: none

Flow control: none

After the system starts up, you can see the following u-boot print:

```
AMP: Brought up primary cpu[f00, self] with state 0x10, entry 0x03900000 ...OK
```

After entering the RT-Thread system, you can see the system information printed:

```
 \ | /
- RT -     Thread Operating System
 / | \     4.1.1 build Oct 21 2024 16:17:13
 2006 - 2022 Copyright by RT-Thread team
Hi, this is RT-Thread!!
msh >I/TC: Secondary CPU 1 initializing
*I/TC: Secondary CPU 2 initializing
*I/TC: Secondary CPU 1 switching to normal world boot
*I/TC: Secondary CPU 2 switching to normal world boot
```